Харківський національний університет імені В.Н. Каразіна

Кафедра інтелектуальних програмних систем і технологій

Дисципліна: Методи контейнерізації та віртуалізації програмних рішень

ЛАБОРАТОРНА РОБОТА №5

Розробка базового Azure DevOps CI pipeline.

Харків 2026 рік

Зміст

Опис 3
Мета лабораторної роботи 3
Схема оцінювання 3
Запит до MS на розблокування AzurePipelines 5
Завдання 6
Завдання №1 [I]. Створення Azure DevOps Pipeline для обраного сервісу 6
Завдання №2 [K]. Автоматичне розрахування версії артефактів 8
Завдання №3 [I]. Сборка проекту та запуск Unit Tests 10
Довідка 10
Додаток А. Версіювання та взаємодія з Docker Hub 11
Група зміних для проекту 11
Робота з Docker Hub API 12
Встановлення значення змінної на етапі запуску Pipeline 16
Список додаткових джерел інформації 21

Опис

Мета лабораторної роботи

1. Отримання практичних навичок в розробці Azure DevOps Pipeline.
2. Вдосконалення навичок роботи з дистрибутивною системою контролю версій – Git.
3. Планування подальшої роботи.

Схема оцінювання

Завдання	Критерії прийняття	Бал
Завдання №1 [I]. Створення Azure DevOps Pipeline для обраного сервісу	• Продемонструвати, що CI pipeline містить stages: pre-build, build & test та post-build • Продемонструвати, що CI pipeline тригериться при змінах в гілці «main» та виконуються всі stage • Продемонструвати, що CI pipeline тригериться при змінах в гілці «feature/*» та виконуються всі stages окрім останньої (post-build) • Продемонструвати наявність кроку, що роздруковує значення зміних середовища	1
Завдання №2 [K]. Автоматичне розрахування версії артефактів	• Продемонструвати роботу з variable group • Продемонструвати, що версія розраховується автоматично • Продемонтсрувати, роботу з змінною, що встановлюється вручну та примусово встановлює версію • Продемонструвати роботу з додатковим репозіторієм	2
Завдання №3 [I]. Сборка проекту та запуск Unit Tests	• Продемонсрувати сборку проєкту • Продемонструвати репорт по запущеним unit tests	1

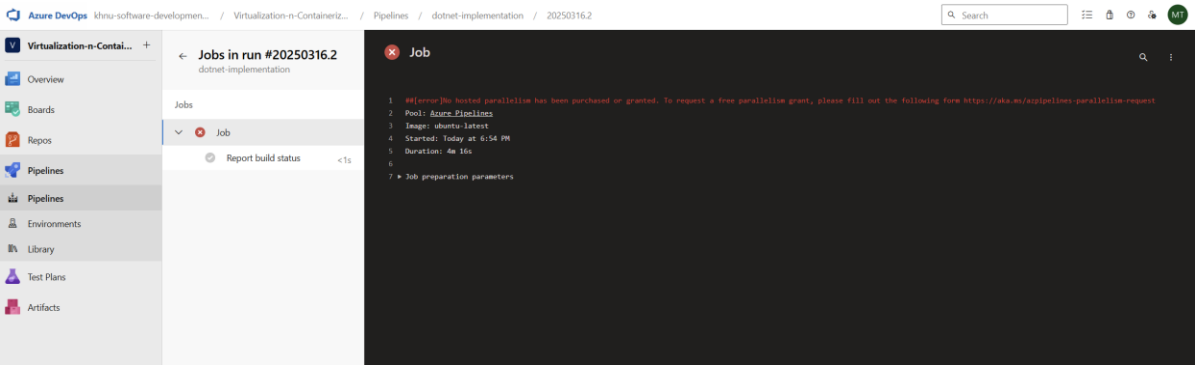
[K] – колективне завдання, оцінку отримує вся команда в незалежності від того, хто скільки зусиль доклав. [I] – інивідуальне завдання, оцінку отримує конкретний виконавець.

Запит до MS на розблокування AzurePipelines

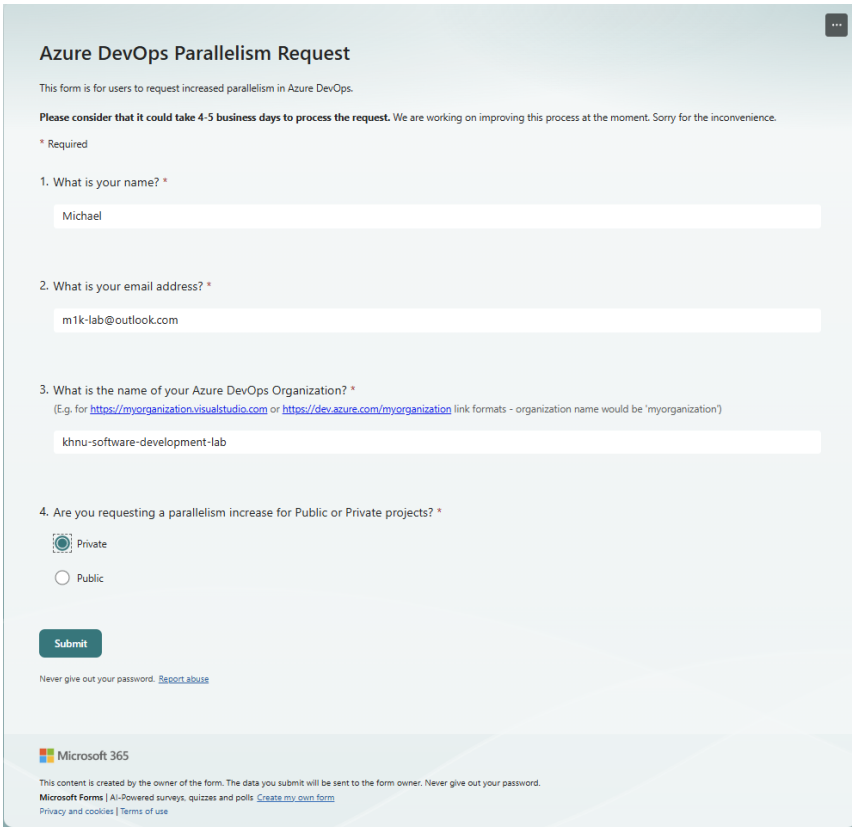
При створенні першого Azure DevOps Pipeline ви можете зіштовхнутись з проблемою, що цей функціонал заблокован.

Помилка буде наступна:

“No hosted parallelism has been purchased. To request a free parallelism grant, please fill out the following form: …”



Вам потрібно перейти за поиланням, вказаним в повідомені на заповнити форму:



Після нажміть Submit.

Обробка займає 1-2 дні, до цього ви НЕ зможете будувати Azure DevOps Pipeline. Тому зробить запит якомога раніше.

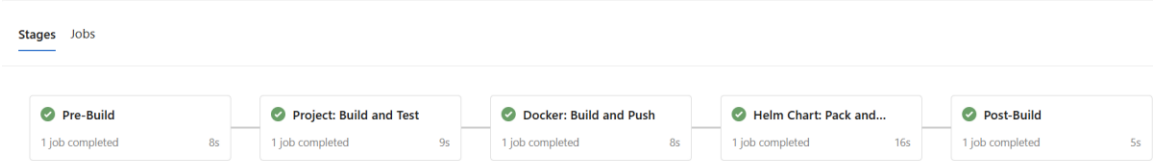
Завдання

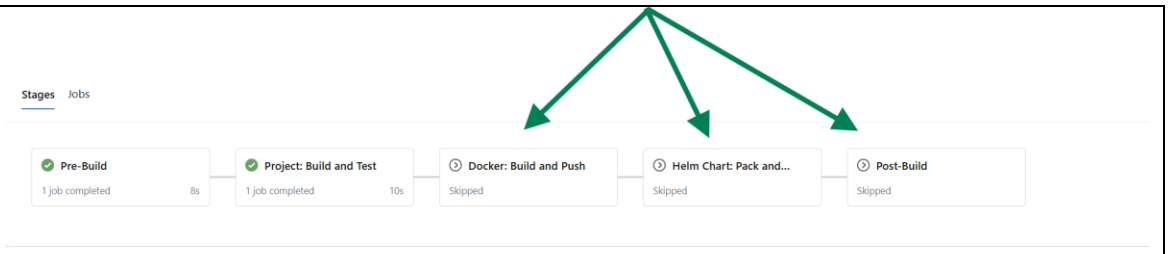
Завдання №1 [I]. Створення Azure DevOps Pipeline для обраного сервісу

В контексті даної задачі ми створимо базовий Azure DevOps Pipeline для нашого сервісу, який буде по різному реагувати на різні гілки (branch) та опишемо декілька функціональних кроків.

№	Опис

A1	Створити Azure DevOps Pipeline, щощо має наступну структуру:
	
	Stage (ім'я – Pre-Build)
	- Job (ім'я – Pre-Build Analysis)
	Stage (ім'я – Project: Build and Test)
	- Job (ім'я – Build And Test)
	Stage (ім'я – Post-Build)
	- Job (ім'я – Post-Build)
	
	Кожна job може мати одну таску, що виведе назву поточного кроку, наприклад:
	
	`- Script |`
	`echo "Hello, from Pre-Build Job"`

A2	Кожна stage повинна виконуватись послідовно і тільки у випадку, якщо попередня завершилась успішно. Якщо в однієї з stage чи job виникла помилка, то pipeline закінчує роботу. Приклад: 
A3	Якщо трігером для pipeline є гілка 'main', то повинні виконуватись всі stage, в випадку коли трігером для pipeline є гілка, що починається на 'feature/*', то повинні виконуватись лише перші дві stage: Pre-Build -> Project: Build and Test В прикладі ви можете побачити, що для гілкі «feature/*» тільки перші 2 stage відпрацювали :

	
A4	Додати step з назвою «Predefined variables» в Pre-Build (stage) -> Pre-Build Analysis (job), і якому роздрукувати (echo) значення наступних зміних середовища: - $(Agent.TempDirectory)" - $(Build.ArtifactStagingDirectory)" - $(Build.Repository.LocalPath)" - $(Build.Repository.Name)" - $(Build.SourceBranch)" - $(Build.SourceBranchName)" - $(Build.SourcesDirectory)" - $(Build.StagingDirectory)"

Завдання №2 [К]. Автоматичне розрахування версії артефактів

В контексті цього завдання ми будемо працювати з першим stage – «Pre-Build».

```
…
- stage: StageID_PreBuild
  displayName: Pre-Build
  jobs:
  - job: JobID_PreBuild
    displayName: 'Pre-Build'
    steps:
…
```

№	Опис
Б1	Створити групу зміних для проекту та додати туди наступні значення: • Docker Id та Token – для доступу до Docker Hub (параметри, що використовуються в команді docker login) • Ім'я Docker Repository • Ім'я сервісу Приклад та пояснення в Додаток А. Версіювання та взаємодія з Docker Hub
Б2	• Створити додатковий git репозіторій з ім'ям "ci-templates" • В репозіторії "ci-templates" створити директорію «common-scripts» і розташувати в неї bash скріпт з ім'ям «logging.sh» який повинен мати: - функцію info_msg(), що отримує один аргумент – строку та виводе її зеленим - функцію warning_msg(), що отримує один аргумент – строку та виводе її жовтим - функцію error_msg(), що отримує один аргумент – строку та виводе її червоним • Додати репозіторій "ci-templates" як ресурс в Pipeline та в степу, що роздруковує predefined variables роздрукувати ці змінні за допомогою функції info_msg з logging.sh (щоб текст був кольоровий)
Б3	Розробити step з ідентифікатором CalculateAppVersion, в контексті job з ідентифікатором JobID_PreBuild: • який вичитає останню доступну версію Docker Image у вказаному Docker Registry (наприклад, m1klab/users-api:2.3.4 – це остання доступна версія, тобто система поверне значення 2.3.4), якщо ще не усніє образів в Docker Registry, функція повинна повернути значення 0.0.0 • код, що визначає доступну версію, винести в окремий bash скрипт, назвати «dockerhub.sh» та розташувати в репозіторії «ci-templates» в диреткорії «common-scripts» • збільшіть PATCH частину повернутого значення на одиницю (тобто стане 2.3.5 чи 0.0.1) • роздрукує нове значення за дпомогою функції info_msg • збереже результат в загальнодоступній змінной на ім'я APP_VERSION Приклад та пояснення в Додаток А. Версіювання та взаємодія з Docker Hub

Б4	Додати можливість примусово використовувати вказану версію для сервісу та не запускати механізм автоматичного розрахунку версії у разі коли версія вказана примусово. Приклад та пояснення в Додаток А. Версіювання та взаємодія з Docker Hub

Завдання №3 [I]. Сборка проекту та запуск Unit Tests

В контексті цього завдання ми будемо працювати з другим stage – «Project: Build and Test»

```
...
- stage: StageID_BuildAndTestProject
  displayName: 'Project: Build and Test'
  dependsOn: StageID_PreBuild
  jobs:
  - job: JobID_BuildAndTest
    displayName: Build and Test
    steps:
...
```

№	Опис
B1	Розробити step, що буде робити build сервіса
B2	Розробити step, що буде робити запуск Unit Test, що відносяться до проекту

Довідка

Розібратись як саме зібрати проект та запустити Unit Tests буде вашою самостійною роботою.

В інтернеті велика кількість прикладів як це зробити для різних мов програмування. Нижче ви знайдете деякі корисні посилання, але не зупиняйтесь тільки на них – спробуйте пошукати відповіді самостійно.

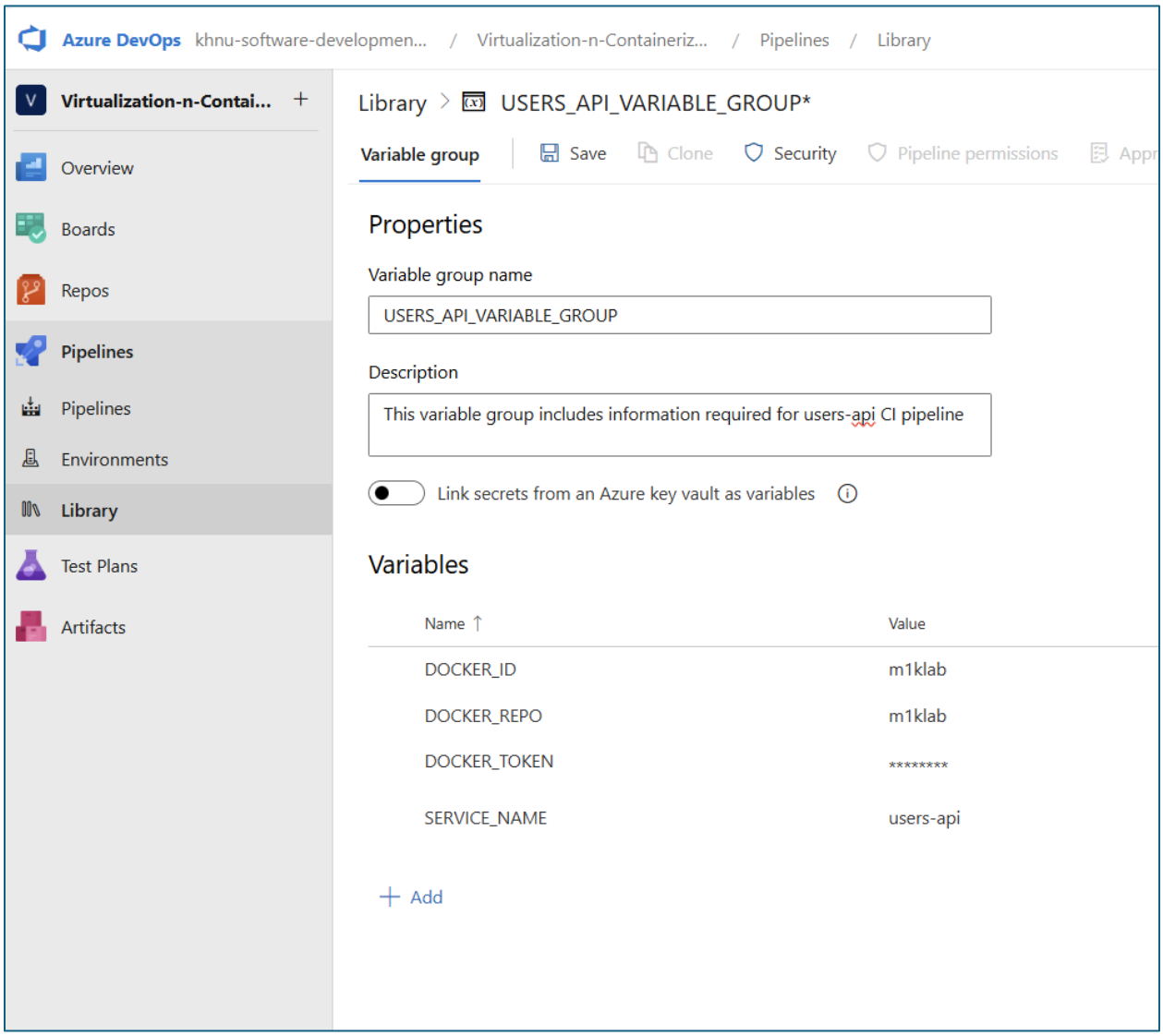
1. Build and publish a Python app
2. Build and publish a Node.js package
3. Build Java apps
4. Build, test, and deploy .NET Core apps

У випадку, якщо сборка пройшла з помилками чи тести не відпраювали CI pipeline повинен бути в стані failed.

Додаток А. Версіювання та взаємодія з Docker Hub

Група зміних для проекту

Групу міних можна створити в Pipelines -> Library:



Як вказано в завданні необхідно створити декілька зміних.

Для того, щоб використовувати ці змінні в azure-pipeline.yaml файлі необхідно задекларувати секцію variables, як покаано нижче:

```
variables:
  - group: 'USERS_API_VARIABLE_GROUP'
  - name: DockerId
    value: "$(DOCKER_ID)"
  - name: DockerToken
    value: "$(DOCKER_TOKEN)"
  - name: DockerRepo
    value: "$(DOCKER_REPO)"
  - name: ServiceName
    value: "$(SERVICE_NAME)"
```

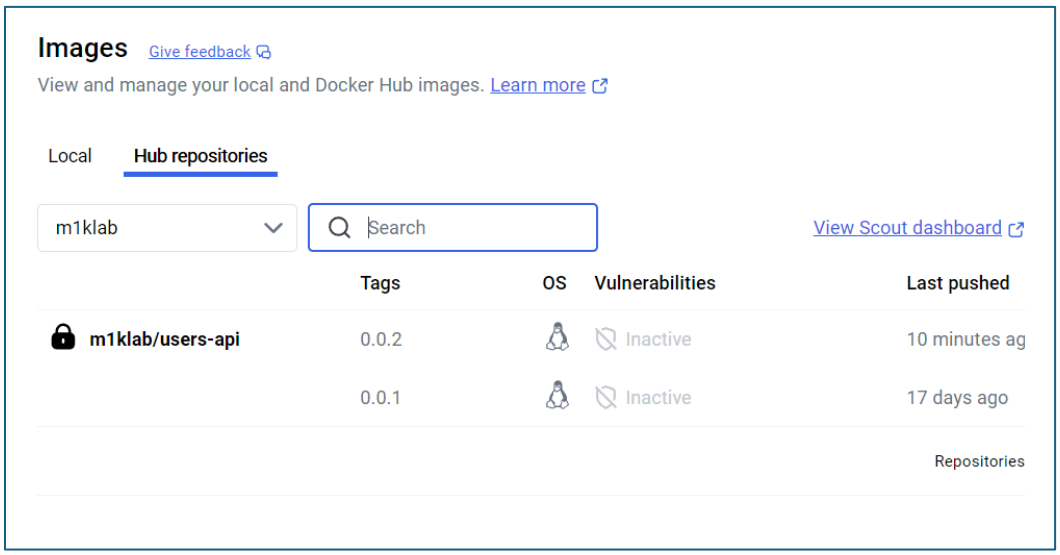
Доступ до значень цих зміних можна отримати за домогою контструкції $(<ім’я змінної>):

```
- script: |
    # use shared functionality
    source "$(Build.SourcesDirectory)/ci-templates/common-scripts/logging.sh"
    info_msg "Info message $(DockerId)"
    info_msg "Info message $(DockerToken)"
    info_msg "Info message $(DockerRepo)"
    info_msg "Info message $(ServiceName)"
```

Робота з Docker Hub API

Час від часу ми вимушені працювати напряму з API того чи іншого Container Registry, тому що не завжди CLI (command line interface) надає нам необхіну функціональність.

Ви можете знайти приклад функції, що отримує останню версію Docker Image нижче. Вхідні данні наступні:



Зверніть увагу, що API Docker Registry могло змінитись, тому у разі коли ця функція не відпрацьовує – доробіть її самотужки.

```
# Description:
# $1 - repository
# $2 - DockerHub username
# $3 - DockerHub password
get_docker_image_latest_version() {
    local repository=$1
    local user=$2
    local password=$3

    # Make a POST request to the Docker Hub login endpoint
    response=$(curl -s -H "Content-Type: application/json" -X POST -d "{\"username\": \"${user}\",
\"password\": \"${password}\"}" https://hub.docker.com/v2/users/login/)

    # Extract the token from the response
    token=$(echo $response | jq -r .token)

    # Check if the token was successfully extracted
    if [ "$token" == "null" ]; then
        # Login failed. Please check your username and password.
        exit 1
    else
        echo "Login successful"
    fi

    # Fetch the tags from Docker Hub API
    response=$(curl -s -H "Authorization: Bearer $token"
"https://hub.docker.com/v2/repositories/${user}/${repository}/tags")

    latest_name=$(echo "$response" | jq -r '.results | map(select(.content_type == "image")) |
max_by(.last_updated) | .name')

    echo "$latest_name"
}

# приклад використання
get_docker_image_latest_version users-api m1klab <ваш токен>
```

Для розуміння як саме працює функція ви можете спробувати її запустити в bash середовище.

!

На машинах з Windows:

1) встановіть пакет jq за допомогою команди (Power Shell): winget install jqlang.jq
2) запустить Git Bash та вже там запускайте цю функцію

Також раджу з метою експеримента додати:

echo “${response}” – просто, щоб подивитись, що саме вам повертає запит (не забудьте після цього закоментувати цей виклик).

Взагалі запит https://hub.docker.com/v2/repositories/${user}/${repository}/tags поверне щось подібне:


```
{
    "count": 1,
    "next": null,
    "previous": null,
    "results": [
        {
            "creator": 33230925,
            "id": 871539504,
            "images": [
                {
                    "architecture": "amd64",
                    "features": "",
                    "variant": null,
                    "digest": "sha256:410083298a7cb28925c445ff5ef2311902ac94df11adfcc0d9e1560df4d182b6",
                    "os": "linux",
                    "os_features": "",
                    "os_version": null,
                    "size": 93886610,
                    "status": "active",
                    "last_pulled": "2025-03-27T08:12:44.121625459Z",
                    "last_pushed": "2025-03-27T08:12:43.300506432Z"
                },
                {
                    "architecture": "unknown",
                    "features": "",
                    "variant": null,
                    "digest": "sha256:f654b7f4e4163a56ad027b53c4e26c5e87490366a02f47bfc39ca59166121d2d",
                    "os": "unknown",
                    "os_features": "",
                    "os_version": null,
                    "size": 1484,
                    "status": "active",
                    "last_pulled": "2025-03-27T08:12:48.108718958Z",
                    "last_pushed": "2025-03-27T08:12:43.22636286Z"
                }
            ],
            "last_updated": "2025-03-27T08:12:44.531998Z",
            "last_updater": 33230925,
            "last_updater_username": "m1klab",
            "name": "0.0.2",
            "repository": 26690759,
            "full_size": 93886610,
            "v2": true,
            "tag_status": "active",
            "tag_last_pulled": "2025-03-27T08:12:48.110388222Z",
            "tag_last_pushed": "2025-03-27T08:12:44.531998Z",
            "media_type": "application/vnd.oci.image.index.v1+json",
```

```
            "content_type": "image",
            "digest": "sha256:9008aa626901660575e649590d1489d1e8a03c752b72b505f1c9396dac7edcb4"
        },
        {
            "creator": 33230925,
            "id": 861614575,
            "images": [
                {
                    "architecture": "amd64",
                    "features": "",
                    "variant": null,
                    "digest": "sha256:98d1fb0794e964c3665a57ef95d2bb15c74b9f4b0f0bb0fc7d2b354da82c652e",
                    "os": "linux",
                    "os_features": "",
                    "os_version": null,
                    "size": 91452007,
                    "status": "active",
                    "last_pulled": "2025-03-10T07:00:31.677591666Z",
                    "last_pushed": "2025-03-10T07:00:30.866550388Z"
                },
                {
                    "architecture": "unknown",
                    "features": "",
                    "variant": null,
                    "digest": "sha256:f1b02bf1ff80983c19b5f8298fe17d8d9effedbb1db6ae4ecd417c6beefd3f9b",
                    "os": "unknown",
                    "os_features": "",
                    "os_version": null,
                    "size": 1484,
                    "status": "active",
                    "last_pulled": "2025-03-10T07:00:32.919719069Z",
                    "last_pushed": "2025-03-10T07:00:30.895888795Z"
                }
            ],
            "last_updated": "2025-03-10T07:00:32.095766Z",
            "last_updater": 33230925,
            "last_updater_username": "m1klab",
            "name": "0.0.1",
            "repository": 26690759,
            "full_size": 91452007,
            "v2": true,
            "tag_status": "active",
            "tag_last_pulled": "2025-03-10T09:33:04.542636917Z",
            "tag_last_pushed": "2025-03-10T07:00:32.095766Z",
            "media_type": "application/vnd.oci.image.index.v1+json",
            "content_type": "image",
            "digest": "sha256:89041ae6ac0eaf6c38f1ff1e7abb18141378aed3e2a2b8d0350359ac62f467f0"
        }
    ]
}
```

А далі ми працюємо з цим json та отримуємо останню версію, в нашому випадку функція поверне 0.0.2

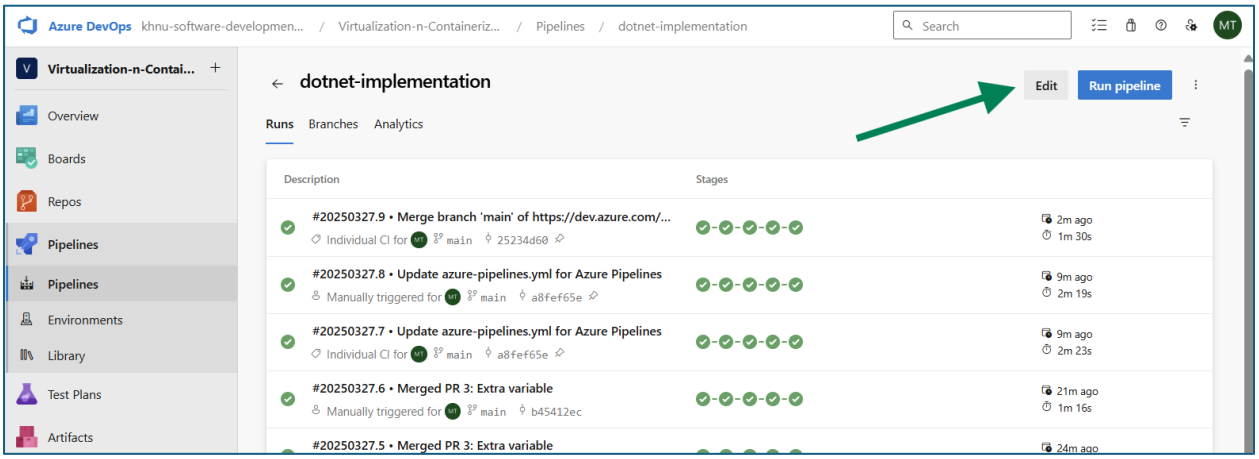
Задача по інкременту PATCH частини винесена на самостійне опрацювання.

Встановлення значення змінної на етапі запуску Pipeline

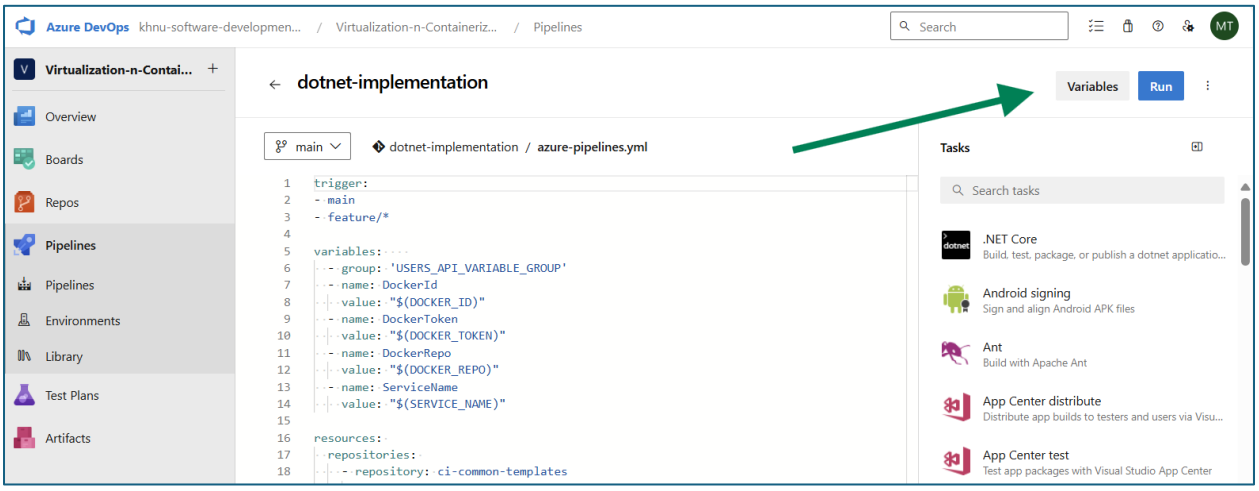
В системі Azure DevOps Pipeline є можливість встановити значення глобальної змінної (що буде доступна в усіх елементах Pipeline) на етапі запуску.

Для цього потрібно зробити наступне:

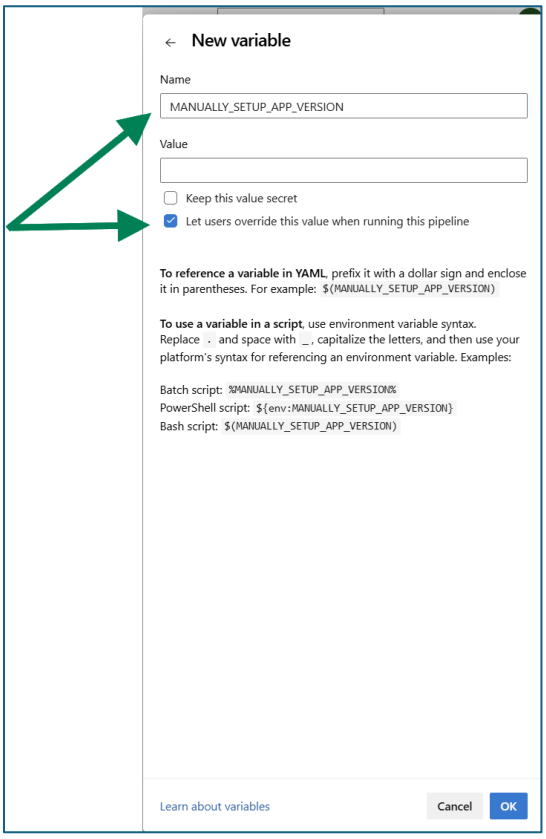
1. Відкрити Pipeline для редагування



2. Нажати на кнопку «Variables»



3. Ввести значення нової змінної:



Після створення цю змінну можна буде використовувати в azure-pipeline.yaml файлі, як будь-яку іншу предвстановлену зміну.

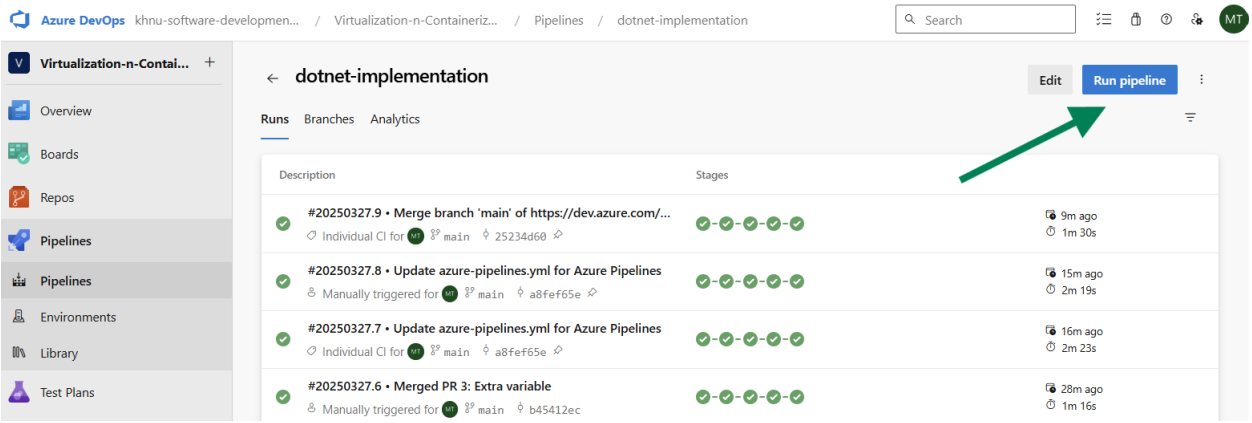
```
- script: |
    # use shared functionality
    source "$(Build.SourcesDirectory)/ci-templates/common-scripts/logging.sh"
    info_msg "Info message $(DockerId)"
    info_msg "Info message $(DockerToken)"
    info_msg "Info message $(DockerRepo)"
    info_msg "Info message $(ServiceName)"

    if [ -z "$(MANUALLY_SETUP_APP_VERSION)" ]; then
      info_msg "MANUALLY_SETUP_APP_VERSION is not set"
    else
      info_msg "MANUALLY_SETUP_APP_VERSION is set to $(MANUALLY_SETUP_APP_VERSION)"
    fi
```

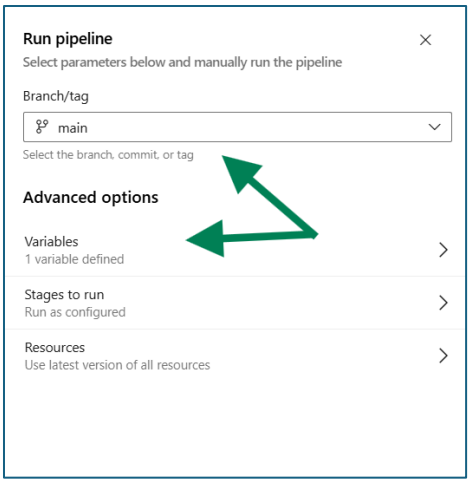
Відмічу, що декларувати її в секції «variables» не потрібно.

При запуску Pipeline вручну ви можете встановити значення для цієї змінної:

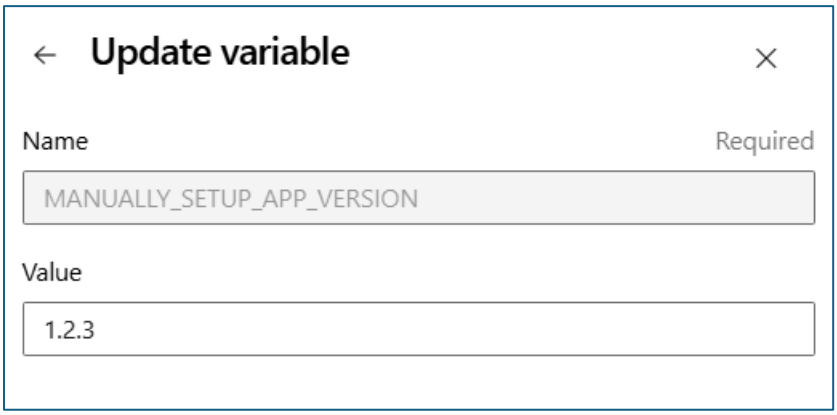
1. Нажимаєм «Run Pipeline»



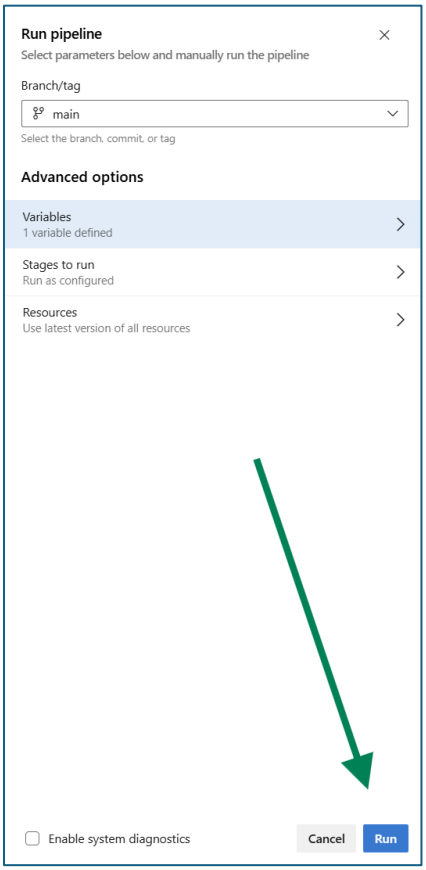
2. Обираєм гілку та нажимаєм на «Variables»



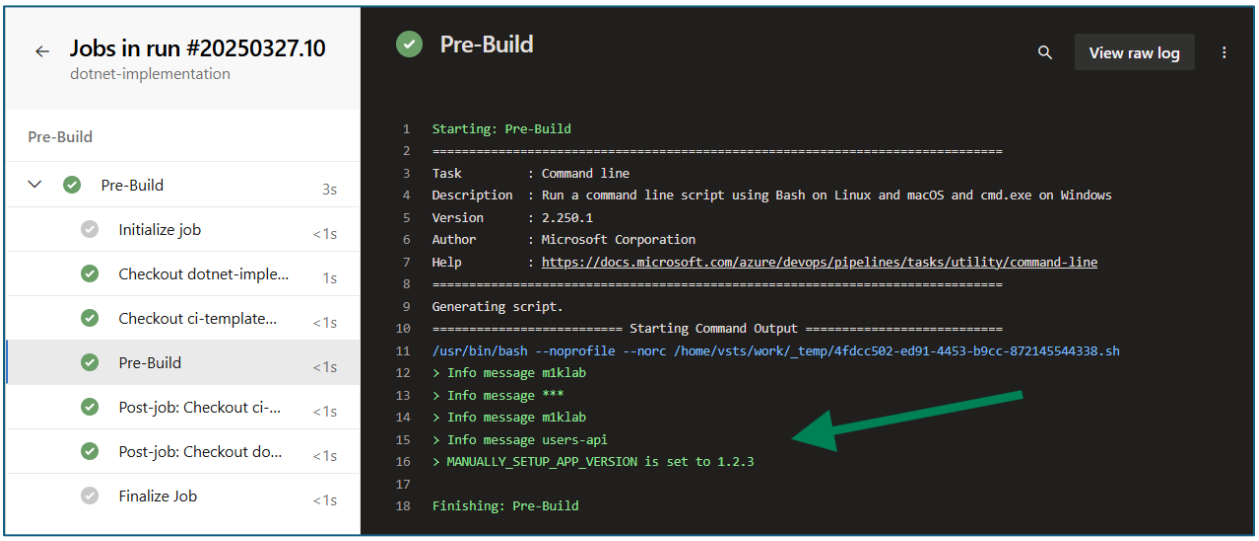
3. Встановлюєм необхідне значення:



4. Запускаєм Pipeline



Результат:



На базі цього функціоналу можна побудувати управління версіями MAJOR та MINOR, наприклад, ви через цю зміну можете одноразово встановити стартову версію – 1.2.0,

а після цього код, що автоматизує отримання версій з Docker Registry буде інкрементувати PATCH чатсину кожен раз коли ви робити зміни в “main” гілці і утворювати версії 1.2.1, 1.2.2, 1.2.3 тощо.

Звичайно існує велика кулькість інших стратегій управління версіями, мабуть навіть більш популярних (наприклад, з використанням git tag), але і цей підхід має право на життя, особливо в невеликих проєктах.

Список додаткових джерел інформації

1. Azure DevOps Pipeline: Key Concepts
2. Stages, Dependencies and Conditions
3. Azure DevOps Pipeline: Expressions